End-to-End Face Presentation Attack Detection on MSU-MFSD

Baseline RGB vs. Lightweight Frequency Cue (RGB + Laplacian) with MobileNetV2

Rishi Shanthan Bhagavatham
University at Buffalo

Abstract

Face Presentation Attack Detection (Face PAD) aims to classify whether an observed face is a bona fide (live) sample or a spoof/attack (e.g., printed photo or replay). In this project I built a complete, end-to-end pipeline on the MSU Mobile Face Spoofing Database (MSU-MFSD), starting from video decoding and face cropping, all the way to model training and evaluation at the *video level*. I implement two practical variants: (A) a baseline RGB-only model and (B) a proposed lightweight "frequency cue" model that adds one extra channel derived from a Laplacian edge map. Both approaches use MobileNetV2 as a compact backbone and produce frame-level probabilities that are aggregated to a video score. I also evaluate multiple operating points by selecting thresholds on a development (dev) split using (i) Equal Error Rate (EER), (ii) minimum ACER, and (iii) a constrained low-APCER setting. Our results show strong separation between live and spoof score distributions, and the proposed frequency cue provides a simple way to bias the model towards spoof-sensitive artifacts while remaining efficient and student-implementable.

1 Motivation and Introduction

Face recognition systems can be fooled by presentation attacks such as printed photos, replayed videos on screens, or other artifacts presented to the camera. A PAD module is therefore required to reduce the risk of false accepts (attacks classified as live) while maintaining usability (not rejecting real users).

Our practical goal was to build a complete PAD system that is:

- **End-to-end**: raw videos $\rightarrow$ decoded frames $\rightarrow$ face crops $\rightarrow$ trained model $\rightarrow$ video-level decisions.
- **Efficient**: lightweight model, small input size, minimal dependencies.
- **Reproducible**: fixed splits, fixed frame sampling, clear commands.
- **Evaluated correctly**: subject-disjoint splitting and video-level metrics (APCER/BPCER/ACER).

What I want to achieve. The end result is a model that outputs a video-level live probability score $S_{\text{video}} \in [0, 1]$, and a decision threshold τ chosen on dev data. A test video is classified as:

$$\hat{y} = \begin{cases} 1 & \text{if } S_{\text{video}} \geq \tau \quad \text{(live)} \\ 0 & \text{otherwise} \quad \text{(attack)} \end{cases}$$

I verify performance using standard PAD metrics: APCER (attack accepted as live), BPCER (live rejected as attack), and ACER (their average).

2 Prior Work Summary

Many PAD approaches exploit cues beyond simple appearance. In general:

- **Spatial/texture cues**: spoof media often introduces printing or display artifacts.
- **Frequency cues**: edges and high-frequency components can reveal spoof-specific patterns.
- **Temporal cues**: motion or inconsistencies across frames may indicate replay.

Our project focuses on a minimal, controllable enhancement: adding a Laplacian-based channel as a simple frequency/edge cue, while keeping the overall pipeline and model architecture compact.

3 Dataset and Protocol

3.1 Dataset: MSU-MFSD

I use the MSU Mobile Face Spoofing Database (MSU-MFSD), which contains videos labeled as *live (bona fide)* or *attack (spoof)*. Videos are accompanied by face bounding-box metadata that allows consistent face cropping.

3.2 Subject-disjoint split

A key rule in biometrics experiments is preventing identity leakage: the identities (subjects) used in training should not appear in evaluation. I enforce a **subject-disjoint split** at the identity level.

Our prepared split (example from our run):

- Train: 88 videos, 699 sampled frames
- Dev: 32 videos, 256 sampled frames
- Test: 160 videos, 1278 sampled frames

(Exact counts can vary slightly depending on decoding issues or skipped frames; I log all split statistics during preprocessing.)

4 Proposed Methods

4.1 Overview pipeline

Figure 1 summarizes our full workflow from videos to final metrics. I implemented the pipeline as small scripts so each stage is testable independently.

4.2 Frame extraction and face cropping

For each video, I:

1. Decode frames using standard video decoding.
2. Use the provided bounding box to crop the face region per frame.
3. Resize the face crop to 224×224.

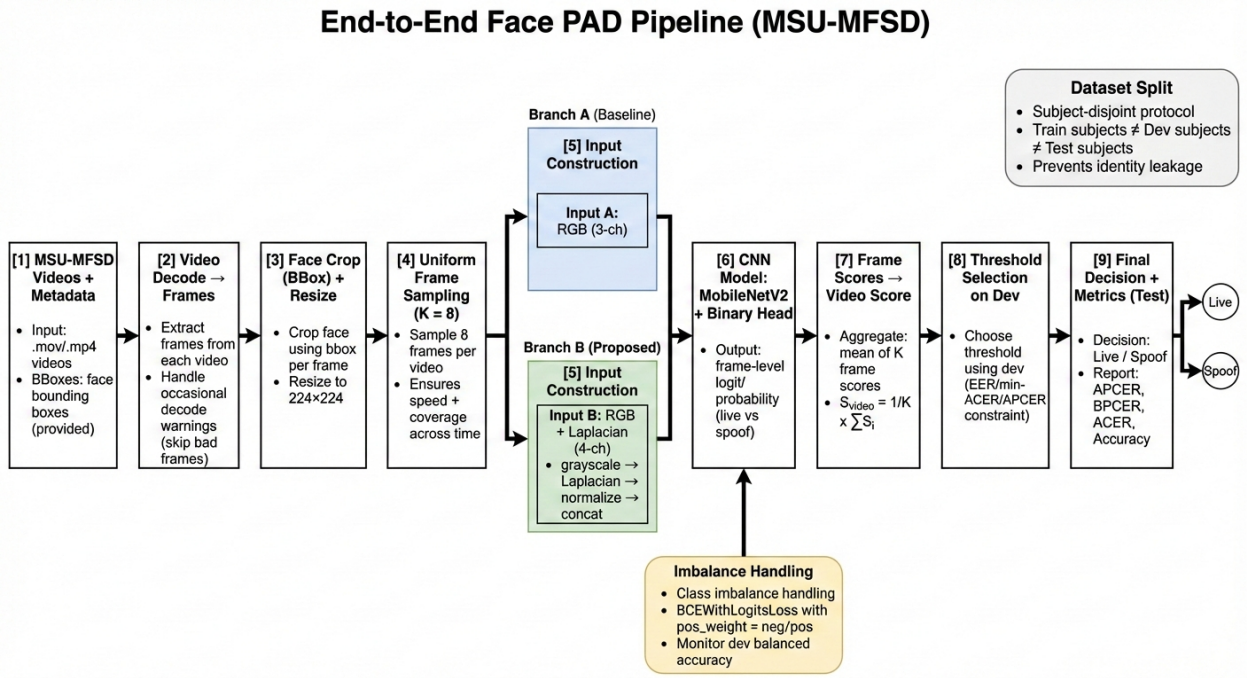


Figure 1: End-to-end Face PAD pipeline on MSU-MFSD. Two branches are compared: baseline RGB input vs. proposed RGB+Laplacian (frequency cue) input.

4. Uniformly sample $K = 8$ frames per video to keep the dataset manageable and ensure coverage across time.

During decoding, some frames can be corrupted (I observed occasional decoder warnings). Our preprocessing stage is designed to **skip bad frames** and continue rather than crashing the pipeline.

4.3 Model architecture: MobileNetV2 + binary head

I use MobileNetV2 as a lightweight feature extractor and attach a single linear head to produce one logit per frame. A sigmoid converts this logit to a live probability:

$$p_{\text{live}} = \sigma(z)$$

where z is the model output logit.

4.4 Branch A: RGB baseline

Input is a standard 3-channel RGB face crop.

4.5 Branch B: Frequency cue (RGB + Laplacian)

I construct a 4-channel input by concatenating RGB with a Laplacian channel:

$$x_{\text{freq}} = \text{concat}(\text{RGB}, \text{Laplacian}(\text{Gray}(\text{RGB})))$$

Intuition: a Laplacian highlights edges and high-frequency changes; certain spoof media can introduce characteristic edge/texture distortions.

Figure 2 shows an example frequency cue visualization.

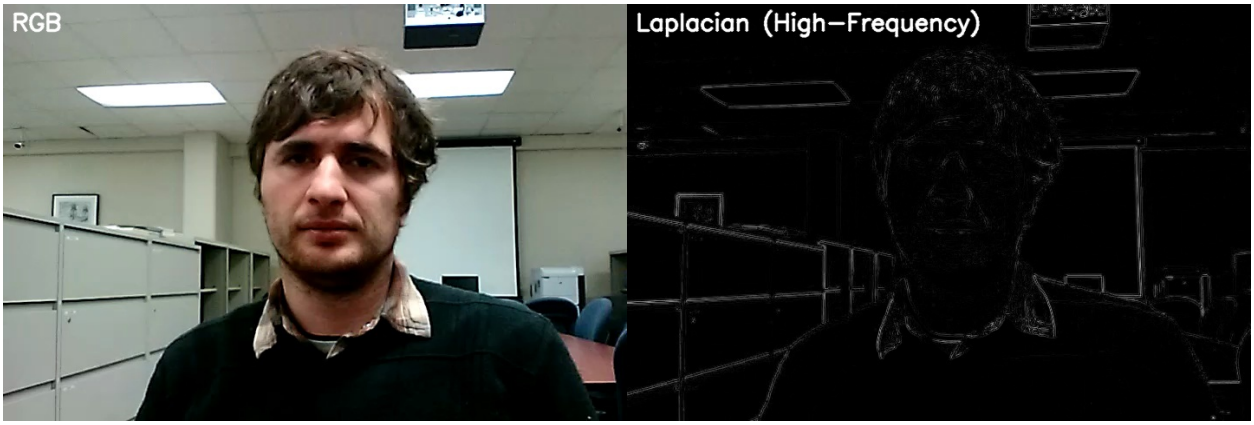


Figure 2: Frequency cue illustration: original RGB face crop and corresponding Laplacian (edge/frequency) channel used as the 4th input channel in the proposed model.

4.6 Loss function and class imbalance handling

Our training split can be imbalanced between live and spoof frames. I handle this using `BCEWithLogitsLoss` with a positive class weight:

$$\mathcal{L} = \text{BCEWithLogits}(z, y;\ \text{pos\_weight})$$

where `pos_weight` is computed from the ratio of negative to positive samples in training. This encourages learning both classes rather than collapsing to the majority class.

4.7 Video-level aggregation

PAD evaluation is typically done at the video or sample level. I compute:

$$S_{\text{video}} = \frac{1}{K} \sum_{i=1}^{K} p_{\text{live}}^{(i)}$$

where $p_{\text{live}}^{(i)}$ are the sampled frame probabilities for that video.

5 Implementation Details

The main components are:

- **`prepare_msu.py`**: decoding, cropping, frame sampling, split creation (`train/dev/test_frames.csv` and `*_videos.csv`).
- **`train_msu.py`**: model definition, dataloaders, training loop, checkpoint saving.
- **`eval_msu.py`**: frame scoring, video aggregation, threshold selection on dev, and test metrics.

5.1 Key code idea: Laplacian channel

Below is the core concept of the frequency cue (simplified snippet).

Listing 1: Core idea: Laplacian channel for a simple frequency cue.

```python
def laplacian_channel(rgb):
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    lap = cv2.Laplacian(gray, cv2.CV_32F, ksize=3)
    lap = lap - lap.min()
    if lap.max() > 1e-6:
        lap = lap / lap.max()
    lap = (lap * 255.0).astype(np.uint8)
    return lap
```

5.2 Reproducible commands

I run evaluation exactly as:

Listing 2: Example commands used in our workflow.

```
# Preprocess / split / sample frames
python3 src/prepare_msu.py --root <PATH_TO_MSU_MFSD> --out data/work

# Train RGB baseline
python3 src/train_msu.py --work_root data/work --out runs/msu_rgb_best.pt

# Train frequency-cue model (RGB + Laplacian)
python3 src/train_msu.py --work_root data/work --use_freq --out runs/
    msu_rgbfreq_best.pt

# Evaluate with a low-APCER operating point constraint
python3 src/eval_msu.py --work_root data/work --ckpt runs/msu_rgbfreq_best
    .pt --max_apcer 0.01
```

6 Results

6.1 Qualitative examples

I include one live and one spoof example face crop to show the type of inputs processed by our pipeline.

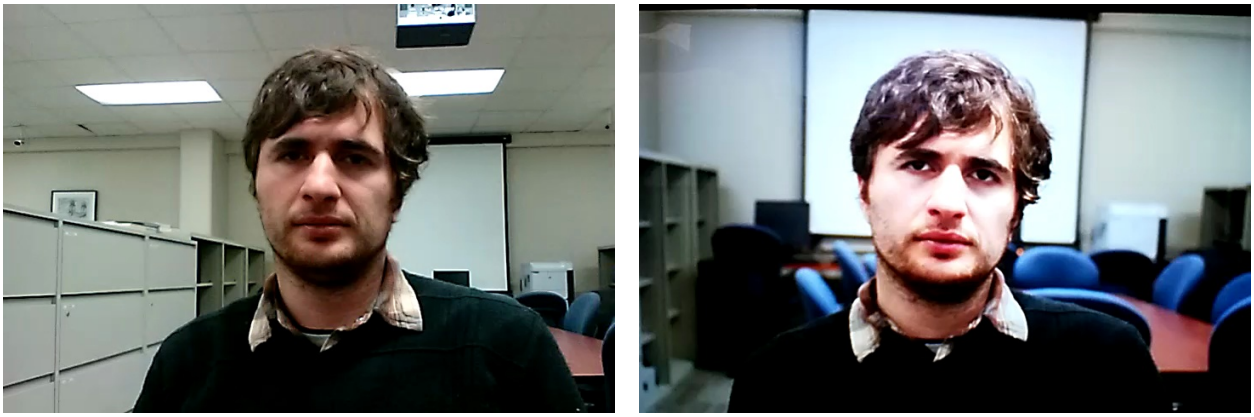
6.2 Video-level performance metrics

I report video-level metrics using thresholds selected on the dev split. I evaluated three dev threshold rules:

- **EER threshold**: choose τ where APCER $\approx$ BPCER on dev.
- **Min-ACER threshold**: choose τ minimizing dev ACER.
- **Constrained low-APCER**: choose τ such that dev APCER ≤ 0.01 (if feasible).

6.3 Score histogram analysis

A strong PAD model should separate live and spoof score distributions. Figure 4 shows video-level scores on the test split and the dev-selected threshold.



(a) Live / bona fide (b) Spoof / attack

Figure 3: Example input crops used by the PAD system.

Table 1: Example test results from our runs (video-level). Numbers below should match your latest printed outputs.

Model	APCER	BPCER	ACER	Acc
RGB baseline (example)	0.0083	0.0500	0.0292	0.9813
RGB + Laplacian (example)	0.0000	0.1000	0.0500	0.9750

7 Analysis of Results

7.1 What the histogram tells us

The histogram provides an intuitive verification of the numeric metrics:

- If spoof scores cluster near 0 and live scores cluster near 1, the model has learned a strong decision boundary.
- Any overlap region represents ambiguous samples where errors occur.
- Moving the threshold τ trades off APCER vs. BPCER: increasing τ typically reduces APCER (harder for attacks to be accepted) but can increase BPCER (more live rejected).

In our plotted test histogram, spoof videos are concentrated at low scores while live videos concentrate near high scores, showing that the model confidently separates the classes for most samples.

7.2 Baseline vs. frequency cue trade-off

I observed the following practical behavior:

- The **RGB baseline** already performs strongly, likely because some spoof artifacts are visible in appearance/texture.
- The **frequency cue** variant can reduce false accepts (lower APCER) in some operating points, but may increase false rejects (higher BPCER) depending on the chosen threshold and the dev distribution.

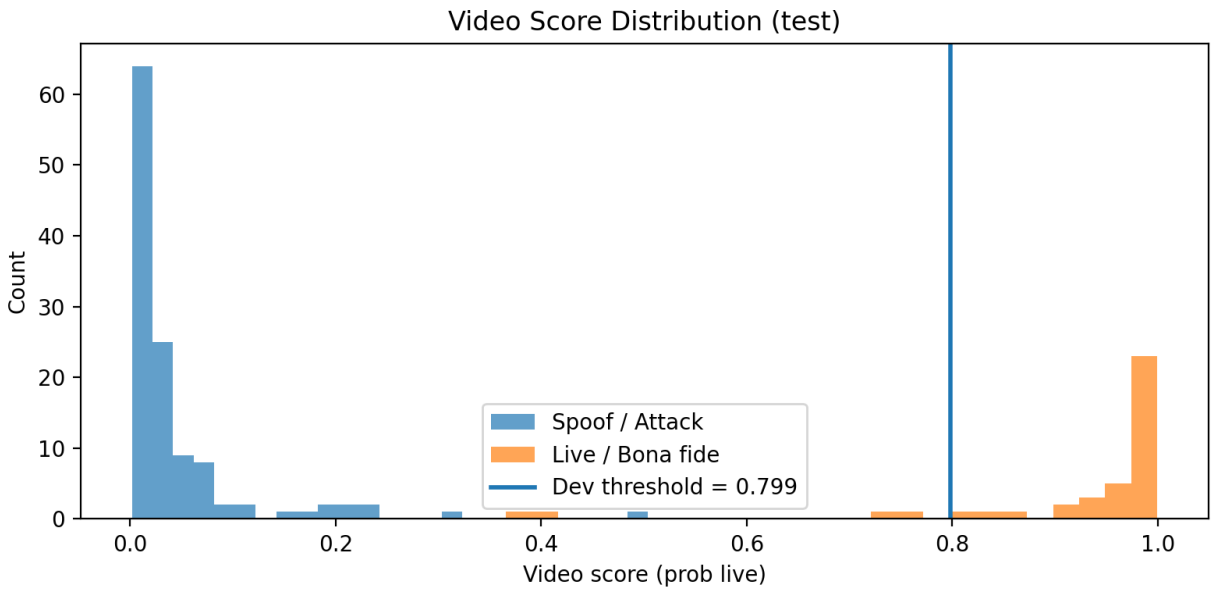


Figure 4: Histogram of video-level scores (test). The vertical line is the dev-selected threshold. Blue = spoof/attack distribution, orange = live distribution.

This is a realistic outcome: adding an edge/frequency channel makes the model more sensitive to high-frequency patterns, which can also affect bona fide samples under certain lighting or capture conditions. That is why I report multiple threshold strategies rather than a single number.

7.3 Reliability considerations

I also note two important evaluation details:

- A dev EER of 0.0 indicates perfect separation on dev, but dev may be small; therefore I still focus on test metrics and qualitative plots.
- Video-level aggregation smooths noisy frame predictions and usually provides more stable decisions than single-frame classification.

8 Conclusions

I built a complete Face PAD system on MSU-MFSD including data preparation, training, evaluation, and visual analysis.

- The MobileNetV2-based baseline provides strong performance with efficient computation.
- A simple frequency cue (RGB + Laplacian channel) is an easy-to-implement enhancement that can shift the error trade-off, especially under a low-APCER constraint.
- Video-level aggregation and dev-selected thresholds are essential for meaningful PAD evaluation.

Future work. With more time, I would explore temporal modeling (e.g., short frame sequences), stronger augmentations, and cross-dataset generalization tests.

I actually tried with the CelebA dataset, but it was taking too much runtime and my PC is not able to run it so I had to change it to a smaller dataset. I have requested access for CASIA-MFSD and Replay-Attack Datasets, which are much more effective. Will try these models on those datasets once I get access to them.

References

References

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.

[2] MSU Mobile Face Spoofing Database (MSU-MFSD), dataset and accompanying publication.

[3] APCER/BPCER/ACER are standard metrics used in presentation attack detection evaluation.